Luca Danisi **21-3-2026**

XSS + SQL Injection

1 - Objective : To test and successfully exploit **Reflected XSS** and **SQL Injection** vulnerabilities in DVWA, showing how insecure input handling can expose session data and sensitive database information.

🧪 Our lab environment

🐉 **Kali Linux**: Attacker machine
💀 **DVWA** : target web application hosted on Metasploitable 2

The two systems were connected through an internal network, with Kali Linux configured as 192.168.50.100 and the target as 192.168.50.101.

2 – Tools Used :

- **Kali Linux** – used as the attacker machine to access DVWA and perform the exploit tests.

- **Metasploitable 2 / DVWA** – used as the vulnerable target machine hosting the Damn Vulnerable Web Application.

- **Web Browser** – used to interact with DVWA and submit XSS and SQL Injection payloads.

- **DVWA (Damn Vulnerable Web Application)** – the intentionally vulnerable web application used for the lab.

- **MySQL** – the backend database exposed during the SQL Injection testing.

3 - Results :

DVWA Exploit ✅ → The exploitation was successful for both vulnerabilities. :

Reflected XSS allowed JavaScript execution in the browser and exposed the session cookie. SQL Injection revealed database error messages, identified backend information, and allowed the extraction of usernames and password hashes from the database

4 - Issues Encountered During the Test:

No major issues were encountered during the lab.
The main difficulty was correctly interpreting the application's responses and adjusting the payloads to confirm the vulnerabilities step by step.

5 – Conclusion :

This lab demonstrated how **Reflected XSS** and **SQL Injection** can be successfully exploited in a vulnerable web application when user input is not properly validated or sanitized. Both Reflected XSS and SQL Injection were successfully exploited, showing how an attacker may expose session data and retrieve sensitive database information.
This highlights the importance of proper input validation and secure coding practices in web applications.

Technical Walkthrough

1. ## Connectivity Check

   A connectivity test was performed before the exploitation phase. The attacker machine, **Kali Linux (192.168.50.100)**, successfully reached the target machine, **Metasploitable 2 / DVWA (192.168.50.101)**, confirming that the lab environment was properly configured and ready for testing.

2. ## Access to DVWA

   Using the browser on Kali Linux, the DVWA instance hosted on the target machine was accessed through http://192.168.50.101. The application was accessed with the default credentials and the security level was set to **Low** in order to reproduce the vulnerabilities in a controlled lab environment.

3. ## Reflected XSS Verification

   The **Reflected XSS** functionality in DVWA was tested by submitting a JavaScript payload that triggered an alert box in the browser. The successful execution of the payload confirmed that user input was reflected back to the page without proper sanitization, allowing arbitrary client-side script execution.

4. ## HTML Injection Confirmation

5. To further confirm the lack of output sanitization, a simple HTML payload was submitted. The application rendered the injected text in italic format, showing that user-controlled input was processed directly by the browser instead of being safely escaped. This behavior further confirmed the presence of the XSS vulnerability.

6. ## Session Cookie Disclosure via XSS

   A second XSS payload was used to display document.cookie inside the browser. This revealed the active session cookie, showing how the vulnerability could expose sensitive session data. In a real-world scenario, this kind of issue could be used for **session hijacking** and user impersonation.

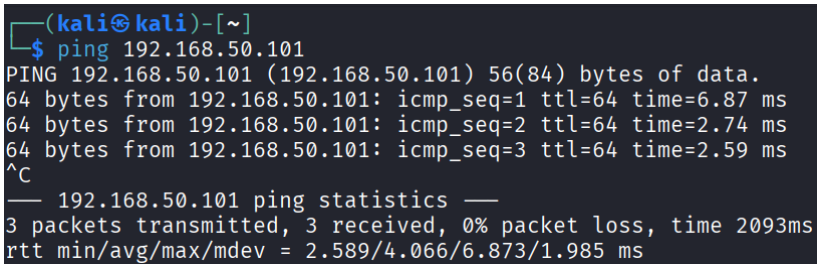
7. ## SQL Injection Exploitation

   The SQL Injection test started by submitting a single quote in the **User ID** field, which generated a SQL error and confirmed improper input filtering. A valid ID was then used to understand the normal structure of the output and identify the number of returned

columns. After that, a **UNION SELECT** payload was used to extract the database user and MySQL version, revealing that the service was running as root@localhost. Finally, another UNION-based payload was used to retrieve usernames and password hashes from the users table, demonstrating successful backend data extraction.

Screenshot captions

Figure 1 – Connectivity check between Kali and Metasploitable



```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=6.87 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=2.59 ms
^C
--- 192.168.50.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2093ms
rtt min/avg/max/mdev = 2.589/4.066/6.873/1.985 ms
```

Figure 2 – DVWA Login and Security Level Configuration

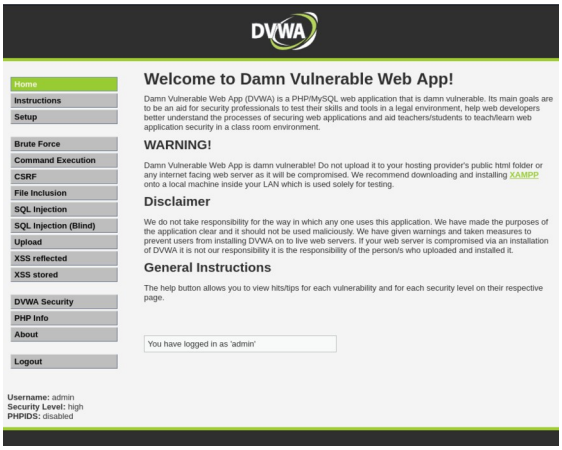


Figure 3 – Reflected XSS Alert Execution

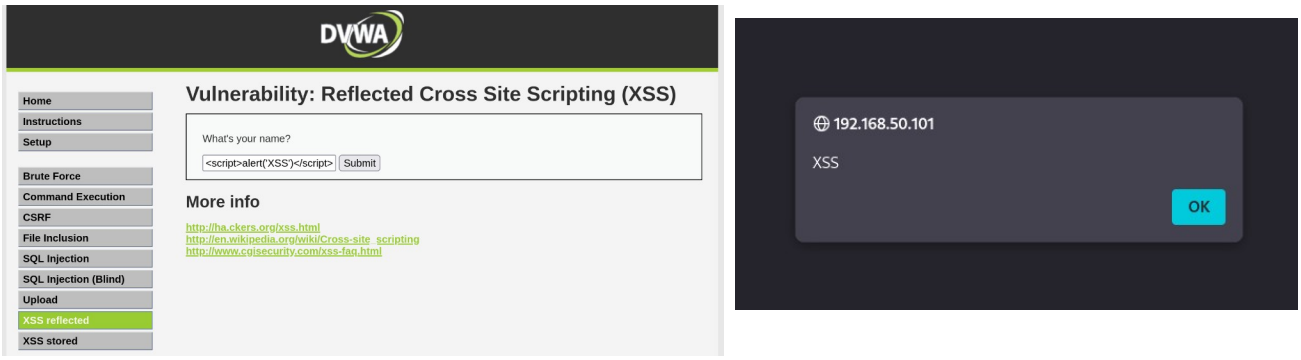


Figure 4 – Session Cookie Disclosure via XSS

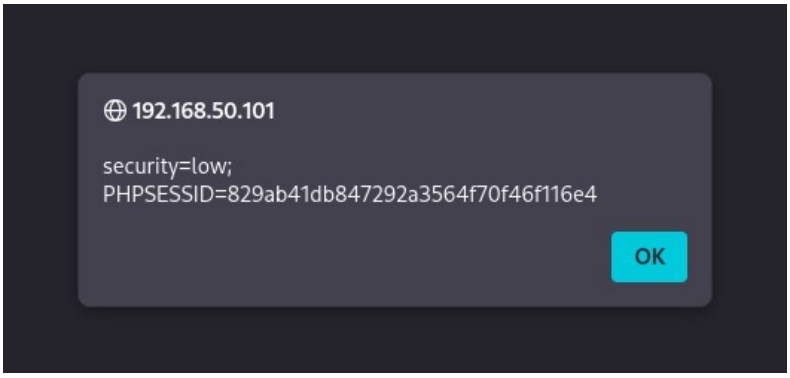
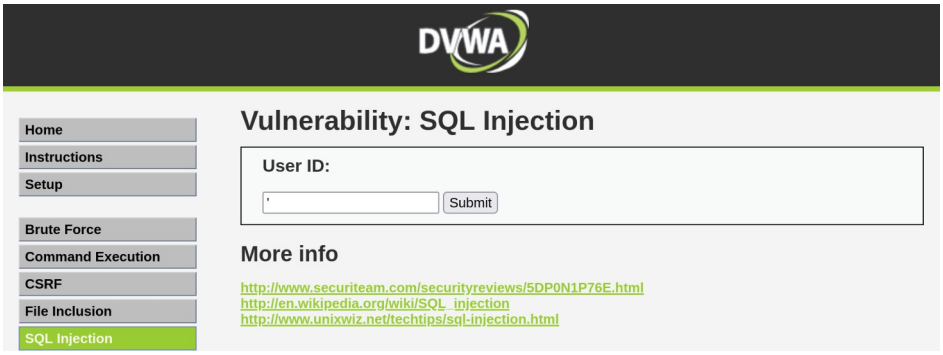
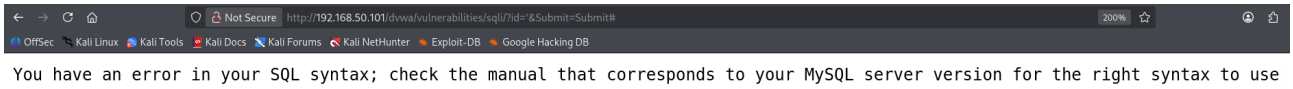


Figure 5 – SQL Injection Error-Based Confirmation






```
You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use
```

End